Structured API - PySpark

RDD to DataFrame

Why Spark Started with RDDs

- Spark was created to overcome **MapReduce limitations** (disk-heavy, slow for iterative workloads).
- RDDs introduced:
    - In-memory computation
    - Lineage-based fault tolerance
    - Fine-grained control over transformations
- Initially focused on **engineering-style distributed computation**, not analytics.

Strength of RDDs

- Flexible
- Powerful
- Suitable for unstructured data
- Developer-controlled execution

However, as Spark moved into analytics-heavy industries, limitations appeared.

Why RDDs Became a Bottleneck in Real Systems

As adoption increased, Spark was used for:

- BI dashboards
- Reporting
- SQL-style analytics
- Business intelligence pipelines

Problems Observed

1. **No Schema Awareness**
    - Spark treats RDD elements as opaque objects.
    - No column understanding.
    - No type awareness.
2. **No Automatic Optimization**
    - Spark cannot optimize joins, filters, or aggregations.
    - All optimization burden falls on the developer.
3. **Verbose & Repetitive Code**
    - Basic analytics operations required manual map/reduce logic.
    - Teams repeatedly implemented similar logic.
4. **Accessibility Problem**
    - Analysts know SQL, not functional programming.
    - RDD API limited cross-team adoption.

Core Realization

> Spark executed fast, but it did not understand the data it was executing on.

Introduction of DataFrames – A Structural Upgrade

DataFrames changed Spark from:

- "Distributed computation engine"
  to
- "Distributed analytics engine"

What Changed?

Instead of:

- Treating data as arbitrary objects

Spark now:

- Treats data as structured tables
- With named columns
- With defined data types
- With nullability information

Imperative → Declarative Shift

RDD = Imperative Programming

- Developer tells Spark step-by-step how to compute.
- Spark executes exactly as written.
- Very little optimization freedom.

DataFrame = Declarative Programming

- Developer describes **what result is needed**.
- Spark decides **how to compute it efficiently**.
- Spark can:
    - Reorder operations
    - Combine transformations
    - Optimize joins
    - Push filters down

Mental Model

- RDD → “Tell Spark how to cook.”
- DataFrame → “Tell Spark what dish you want.”

This shift unlocks advanced optimization.

Schema – The Foundation of Optimization

A **schema** defines:

- Column names
- Data types

- Nullability

Why Schema Matters

With schema, Spark can:

- Perform column pruning
- Push filters to data source
- Use optimized join strategies
- Apply vectorized execution
- Avoid reading unnecessary columns

RDDs cannot do this because:

- Spark sees them as generic objects.

DataFrames as Logical Plans

DataFrame operations:

- Do NOT execute immediately.
- Spark builds a **logical plan** first.

Flow:

User Code
↓
Lazy Transformations
↓
Logical Plan
↓
Optimized Logical Plan (Catalyst)
↓

Physical Plan
↓
Execution (Tungsten engine)

This holistic planning is impossible with low-level RDD execution.

RDD vs DataFrame – Core Comparison

High-Level Comparison

Feature	RDD	DataFrame
Abstraction	Distributed objects	Structured table
Schema	No	Yes
Programming	Imperative	Declarative
Mental Model	Row-oriented	Column-oriented
Optimization	Manual	Automatic (Catalyst, AQE)
SQL Support	None	Native
Performance	Slower for analytics	Faster for analytics
Accessibility	Engineers	Engineers + Analysts

Row-Based vs Column-Aware Processing

RDD (Row-Based)

- Each record processed individually.
- Spark must deserialize every row.
- Cannot skip unused fields.

Analogy:
Warehouse with unlabeled boxes — must open each one.

DataFrame (Column-Aware)

- Structured table with labeled columns.
- Spark can:
    - Read only required columns
    - Skip irrelevant data
    - Apply vectorized execution

Analogy:
Warehouse with labeled shelves — go directly to required section.

This enables massive performance gains at scale.

Analytics Workloads vs Engineering Workloads

Engineering Workloads

- Custom logic
- Complex control flows
- Object-level manipulation
- RDD is suitable

Analytics Workloads

- Group by
- Aggregations
- Filters
- Joins
- SQL queries

RDDs make analytics:

- Verbose
- Repetitive
- Hard to maintain

DataFrames:

- Express business logic clearly
- Reduce boilerplate code
- Enable reusable execution strategies

Why Structure Matters at Scale

In large systems (retail, finance, telecom):

- Billions of rows
- Dozens of columns

Without schema:

- Must scan every row

- Process unnecessary data

With schema:

- Spark skips unused columns
- Pushes filters to storage
- Reads only relevant partitions

Performance improvement is dramatic.

How Schemas Unlock Optimization

Schema enables:

- Column pruning
- Predicate pushdown
- Optimized join selection
- Vectorized execution
- Efficient compression

Example:
Bank computing deposits per branch:

- Spark reads only branch_id and amount
- Skips unrelated columns
- Applies aggregation efficiently

RDD cannot support this.

Creating DataFrames

A) From In-Memory Data

Used for:

- Testing
- Small lookup tables
- Prototyping

```
data = [
    (1, "Alice", 50000),
    (2, "Bob", 60000)
]

df = spark.createDataFrame(data, ["id", "name", "salary"])
df.show()
df.printSchema()
```

Key Points

- Spark infers types.
- Good for small datasets.
- Not ideal for large production systems.

B) With Explicit Schema (Production Preferred)

```
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType, DoubleType

schema = StructType([
    StructField("id", IntegerType(), False),
```

```
    StructField("name", StringType(), True),
    StructField("salary", DoubleType(), True)
])

df = spark.createDataFrame(data, schema)
```

Why Explicit Schema?

- Prevents wrong type inference
- Ensures data contracts
- Critical in financial/regulated systems

C) From External Sources (Most Common in Industry)

Text Files

```
df_text = spark.read.text("data/sample.txt")
```

- Single column DataFrame (value: string)

CSV Files

Explicit Schema:

```
df_csv =
spark.read.option("header","true").schema(schema).csv("data/employees.
csv")
```

Schema Inference:

```
df_csv =
spark.read.option("header","true").option("inferSchema","true").csv("d
ata/employees.csv")
```

JSON Files

```
df_json =
spark.read.option("multiline","true").json("data/employees.json")
```

- Supports nested structures
- Schema inferred or enforced

Parquet Files (Production Standard)

```
df_parquet = spark.read.parquet("data/employees.parquet")
```

- Columnar storage
- Schema stored in metadata
- Supports pruning & pushdown
- Preferred in Data Lakes

From Databases (JDBC)

```
df_jdbc = spark.read.jdbc(url=jdbc_url, table=table_name,
properties=properties)
```

- Connects to MySQL/Postgres/Oracle etc.
- Supports filter pushdown

- Used in ETL & hybrid environments

Why Schema Structure Is Critical

Schema as a Distributed Contract

- Shared agreement across systems
- Ensures consistent interpretation
- Enables governance & data contracts

Without schema:

- Each system must rediscover meaning of fields
- Leads to inconsistency

Data Types as Metadata

Data types:

- Guide optimization
- Enable efficient execution
- Not just rigid constraints

Example:

- Numeric → vectorized aggregation
- Timestamp → efficient time filtering

Nullable vs Non-Nullable

Schema encodes guarantees.

Example:

```
StructField("transaction_id", StringType(), nullable=False)
```

- Non-nullable → must exist
- Nullable → optional business attribute

Runtime Impact

- Non-nullable fields reduce runtime checks
- Nullable fields require safe handling

Schema Inference vs Schema Enforcement

Schema Inference

- Spark guesses types
- Good for exploration
- Risky in production

Schema Enforcement

- Developer defines schema

- Stable
- Predictable
- Preferred for enterprise pipelines

DataFrames as Declarative Intent

RDD:

- Tightly couples business logic & execution.

DataFrame:

- Separates intent from execution.
- Spark decides optimal strategy.

This allows:

- Catalyst Optimizer
- Adaptive Query Execution (AQE)
- Advanced join strategies
- Runtime statistics usage

Lazy Evaluation in DataFrames

Transformations:

- select()

- filter()
- join()
- groupBy()

These:

- Do NOT execute immediately
- Build a logical plan

Execution occurs only when:

- show()
- count()
- collect()
- write()

Code Example (Lazy Execution)

```
df_filtered = df.filter(df.salary > 55000)
df_selected = df_filtered.select("id", "name")

df_selected.show()
```

Execution happens only at `show()`.

Transformations vs Actions

Transformations

- Lazy
- Build plan
- Return new DataFrame

Examples:

- filter
- select
- join
- groupBy

Actions

- Trigger execution
- Return result or write data

Examples:

- show()
- count()
- collect()
- write()

Full Execution Flow

User Code
↓
Lazy Transformations

↓
Logical Plan
↓
Optimized Logical Plan (Catalyst)
↓
Physical Plan
↓
Execution (Tungsten Engine)

This is the intelligence layer that RDDs lack.

🎯 Key Interview Questions

- Why were DataFrames introduced if RDDs were already powerful?
- Explain imperative vs declarative programming in Spark.
- How does schema enable Catalyst optimization?
- What is column pruning?
- What is predicate pushdown?
- Why is Parquet preferred in production?
- Difference between schema inference and schema enforcement?
- What is lazy evaluation?
- How does Spark build execution plans for DataFrames?

🔁 Final Recap

- RDD = control, flexibility, low-level power.
- DataFrame = structure, intelligence, optimization.

- Schema = foundation for performance & reliability.
- Declarative model enables Catalyst & AQE.
- Lazy evaluation allows global optimization.
- DataFrames made Spark enterprise-ready for analytics.